Proposed Novelty: An Explainable Quality Gate for Metric-Based Code Risk Prediction

Oct 25, 2026 · @Mudassir

Mudasir Ali · Reg. No. 232073 · BSSE-F23(A) · Air University, Islamabad

1. Introduction

This project proposes an explainable, validated quality gate: the metric-based prediction model runs inside the CI/CD pipeline on every change and tells developers, file by file, why code was flagged. It builds on Diouf et al. [1], who show that static quality metrics carry a strong signal of security-relevant defects.

Their study analyses 338,442 source files from seven open-source ecosystems, of which 33,294 are affected by 7,685 confirmed security bugs. Five feature-importance methods converge on a small core of size and complexity metrics, and files involved in security fixes show medians roughly three times higher than clean files. Among eleven classifiers evaluated with time-series cross-validation, XGBoost performs best, with recall 0.82, precision 0.95 and ROC-AUC 0.91.

Our project replicates this methodology on the multi-smell dataset, where the target is code smells rather than security bugs. It then carries the work through a complete engineering workflow: development in VS Code, version control on GitHub, continuous integration and deployment with GitHub Actions, an Angular dashboard on GitHub Pages and a Flask inference API. The replication tests whether the paper's patterns hold; the novelty described in this document is what we add on top of it.

2. Research gap

The paper ends where practice begins: it demonstrates that metrics predict risky files offline, but leaves their use inside a real development workflow as future work. Its conclusion explicitly calls for a lightweight CI/CD integration that operationalises the thresholds and model predictions, and for studies of the cost-benefit trade-off of acting on them [1]. Three concrete gaps follow from this.

Gap 1: no operational integration. Pre-commit hooks, CI/CD gates with tiered risk levels and prioritised review are discussed as deployment strategies (Section 5.3.2 of [1]), but none is built or evaluated. The comparison of related work in the paper's Table 1 covers

data, features, models, validation and thresholds; deployment is not among the dimensions compared.

Gap 2: no per-file explanation. The model returns a label or a probability. The feature-importance tables of RQ1 explain which metrics matter on average across 338,442 files, not why one particular file was flagged. A developer who receives an unexplained warning has little reason to act on it, which feeds the alert fatigue the authors themselves warn about.

Gap 3: thresholds that are described but not validated. The warning and critical thresholds are the median and third quartile of the buggy class (Table 14 of [1]). They are never evaluated as detectors, so their precision, recall and false-alarm rate are unknown. The authors acknowledge that the thresholds have not been validated within active development workflows, and they leave combinations of metrics as an open direction.

Together these gaps mean that the paper's findings cannot yet be adopted by a team as a working, trustworthy check.

3. Proposed novelty

We propose an **Explainable Quality Gate (XQG)**: a pipeline stage that scores every changed file with the trained model, explains each score, and routes the change to one of three validated risk tiers. The novelty is not a new classifier. It is the layer between prediction and decision that the base paper describes but does not build.

The contribution consists of three claims:

1. **Operationalisation.** An end-to-end implementation of the CI/CD integration that Diouf et al. list as future work. The model runs in a GitHub Actions job on every push and pull request, with results shown in the job summary, as file annotations and on the Angular dashboard.
2. **Per-file explanation.** Every flagged file carries its reason in two complementary forms: SHAP contributions [2], [3] that show which metrics pushed this file's score up, and its position against the warning and critical thresholds. A developer sees, for example, that a file was flagged because its summed cyclomatic complexity is above the critical threshold and its executable-statement count contributes most to the score.
3. **Validated tiered decisions.** The paper's descriptive thresholds become evaluated decision rules. Each threshold is measured as a detector on held-out folds, model probability and threshold breaches are combined into tiers, and tier cut-offs are fixed

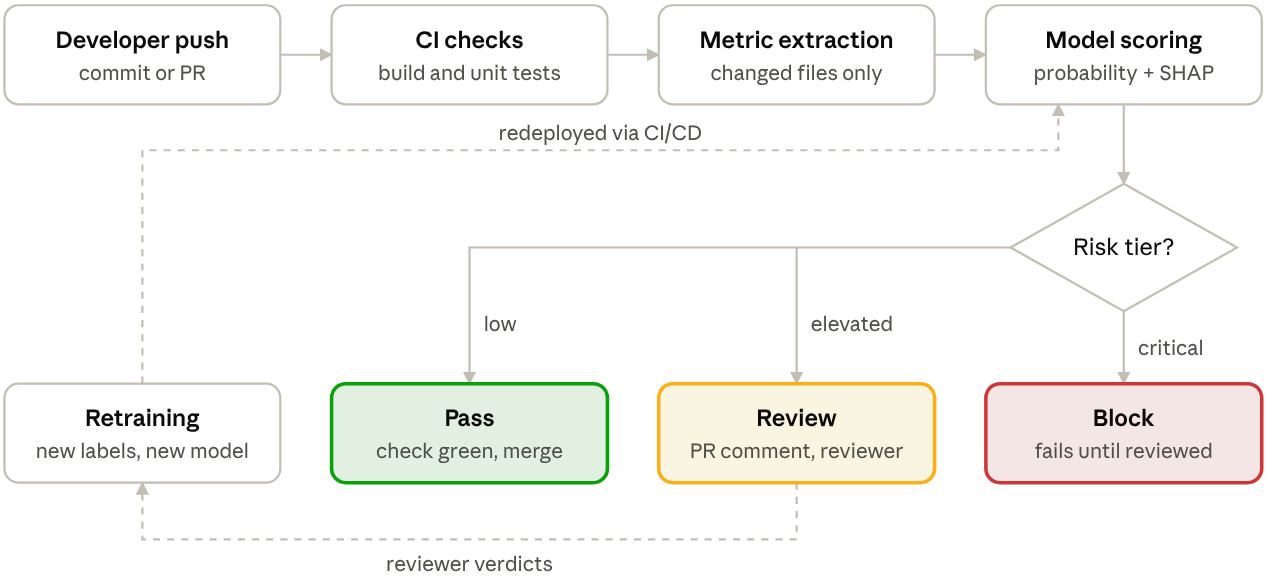
on training data only. The output is a reported precision, recall and review workload per tier, not an assumed one.

In short, the base paper answers *can metrics predict risky code?*; this project answers *can that prediction be trusted and acted on in a team's daily workflow, and at what cost?*

4. Design and method

The gate adds one job to the existing GitHub Actions pipeline, after the build and tests, and loads the model directly in the runner so that no server is needed during CI. The Flask API keeps its role as the interactive backend of the dashboard.

Every push is scored, explained and routed to one of three tiers



Explainable quality gate in the CI pipeline · 4 steps, 1 decision, 3 tiers, 1 feedback loop

Low-risk changes pass silently, elevated ones pass with an explanation on the pull request, and critical ones wait for a reviewer whose verdict feeds the next retraining.

Feature parity. The metrics computed in CI must match the definitions used in the training dataset, otherwise the model scores features it has never seen. The extraction step therefore uses the same metric tool as the dataset where it is available. If it is not, a parity model is trained on the subset of metrics an open-source tool reproduces, and the performance cost of that substitution is reported.

Scoring and explanation. For each changed file the job computes the probability of a smell with the best model from the replication. For tree ensembles such as XGBoost [4], TreeSHAP [3] gives exact per-feature contributions in milliseconds. The three largest contributions and any threshold breaches form a short, readable reason attached to the file.

Tier policy. Probability and threshold breaches are combined into three tiers. The cut-offs are learned on training folds only: the elevated cut-off is set to reach a recall target fixed before the experiment, and the critical cut-off to a high-precision operating point.

Tier	Condition (cut-offs learned on training folds)	Action in the pipeline
Low	Probability below the elevated cut-off and no critical breach	Check passes; nothing is posted
Elevated	Probability above the elevated cut-off, or one critical breach	Check passes with a warning; explanation posted on the pull request
Critical	Probability above the critical cut-off, or two or more critical breaches	Check fails until a reviewer approves; explanation and dashboard link posted

Feedback loop. Reviewer verdicts on flagged files are stored as new labels. The model and the cut-offs are retrained on a fixed schedule, and each retrained version is redeployed through the same CI/CD pipeline, so the gate improves with use instead of drifting.

5. Evaluation plan

The gate is evaluated on the same cross-validation folds as the replication, with SMOTE [5] applied to training folds only, so every number is measured on data the model and the cut-offs never saw. Four questions guide the evaluation, each compared against two baselines: the paper-style thresholds applied alone, and the model probability at a fixed 0.5 cut-off.

Question	Measures	Compared against
N1. Does the tiered gate keep the model's detection quality?	Recall, precision, MCC, PR-AUC per tier	Model at 0.5; thresholds alone

Question	Measures	Compared against
N2. How well do the paper's thresholds work as detectors, and do validated or combined rules do better?	Precision, recall, F1 and flag rate per rule	Median / Q3 rules from [1]
N3. What review workload does the gate create?	Share of files flagged per tier; files reviewed per true smell found; added CI time per run	Thresholds alone
N4. Are the explanations stable and faithful?	Agreement of SHAP top-3 features across folds; overlap between top contributors and threshold breaches	Global RQ1 rankings

The gate is considered successful if, at a recall at least equal to the paper-style thresholds, it flags fewer files, and if its added CI time stays small and is reported per run. Statistical differences between the gate and the baselines are tested with the Wilcoxon signed-rank test over folds.

6. Significance, limitations and conclusion

Significance. For practice, the gate turns a research finding into a check a team can run on every commit, with a reason attached to every warning. For research, it gives a measured answer to three questions the base paper leaves open: how its predictions behave inside a CI/CD pipeline, how good its thresholds really are, and what acting on them costs in review effort.

Limitations. Four limits are acknowledged in advance:

- The gate is validated on code smells, not security bugs, so its numbers do not transfer directly to the paper's setting.
- If the smell labels were produced by metric-threshold rules, the model can partly rediscover those rules; an ablation without the rule-defining metrics is reported to measure this.
- The evaluation is retrospective. A prospective study on live pull requests would be the stronger test and is left as future work.
- SHAP explains the model's behaviour, not the cause of a smell. Explanations support a reviewer's judgement; they do not replace it.

Conclusion. Diouf et al. show that size and complexity metrics predict risky code. This project keeps their method, replicates it on a new dataset, and adds the missing step: an explainable, validated quality gate that runs automatically in CI/CD and reports what it costs. The result moves metric-based prediction from a reported association to a decision a development team can trust and use.

References

1. M. Diouf, E. Toe, M. Grichi, H. Nakouri and F. Jaafar, "Predicting Software Security Bugs Using Machine Learning and Quality Metrics: An Empirical Study," *Computers, Materials & Continua*, vol. 87, no. 3, 2026. doi:10.32604/cmc.2026.077139
2. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.
3. S. M. Lundberg et al., "From Local Explanations to Global Understanding with Explainable AI for Trees," *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020.
4. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
5. N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.